Trait associations by group

Table of contents

Packages 1
Inputs 1
Windows and traits 3
Tests per feature type 3
How many links, against chance 5
Validation 6
Summary 6
Save 7

Do proteins, gene-set scores and module eigengenes track the phenotype (muscle size and strength), and does that link differ between the two groups? The pooled, label-free version of this question lives in `04_Continuous_no_group`; this step asks only what the grouping adds. Two tests:

- Within each group: Spearman's rho between a feature and a trait, with its exact p and a Fisher-z 95% interval. Descriptive only: each group holds 6 to 8 people, and a group split on a trait has a narrowed range of that trait inside it.
- Between groups: limma on feature ~ group × trait. The interaction is the first group's slope minus the second's, with its 95% interval: the one test of whether a link differs between groups. The traits that defined the grouping stay out, since they re-measure the split. Each group needs five people with the trait measured.

Packages

```
library(here)
library(limma)
library(dplyr)
library(tidyr)
library(purrr)
library(tibble)
library(ggplot2)
source(here("00_Setup", "Shared_functions", "analysis_reporting_helpers.R"))
params$grouping
```

```
[1] "Proteome_acute"
```

Inputs

Rank correlations take no covariate, so proteins come from limpa's blood-corrected matrix.

```
data_dir <- file.path("03_Group_comparisons", params$grouping, "c_data")
inputs <- c(
  design = file.path(data_dir, "01_design.rds"),
  corrected = "01_Processing/03_Quantification/c_data/DAList_corrected.rds",
```

```
  singscore = "01_Processing/05_Set_Scores/c_data/singscore.rds",
  gene_sets = "01_Processing/04_Gene_Sets/c_data/gene_sets.rds",
  modules = "01_Processing/06_Module_Building/c_data/modules.rds",
  phenotype = "00_Setup/Input/phenotype.csv"
)
paths <- map_chr(inputs, here)
stopifnot(file.exists(paths))
phenotype <- readr::read_csv(paths[["phenotype"]], show_col_types = FALSE)
d <- readRDS(paths[["design"]])
group_of <- d$group_of
group_levels <- rev(d$groups)
defining_outcomes <- d$defining_traits
dal <- subset_for_design(readRDS(paths[["corrected"]]), d)
catalog <- readRDS(paths[["gene_sets"]])$set_catalog |> filter(qualifies)
modules <- subset_for_design(readRDS(paths[["modules"]]), d)
meta <- as_tibble(dal$metadata)
feature_levels <- list(
  proteins = list(values = dal$data, label = with(dal$annotation, set_names(gene,
uniprot_id))),
  sets = list(
    values = subset_for_design(readRDS(paths[["singscore"]]), d)$scores[catalog$set_id, ],
    label = with(catalog, set_names(pathway, set_id)),
    collection = with(catalog, set_names(collection, set_id))
  ),
  modules = list(values = modules$eigengenes, label = set_names(rownames(modules$eigengenes)))
)
walk(feature_levels, \(level) stopifnot(identical(colnames(level$values), meta$sample_id)))
map(feature_levels, \(level) dim(level$values))
```

```
$proteins
[1] 1900   44

$sets
[1] 1378   44

$modules
[1] 15 44
```

```
defining_outcomes
```

```
character(0)
```

The second group is the reference level, so the group-by-trait term reads as the first group's slope minus the second's (for HR/LR, HR minus LR).

Windows and traits

Each feature enters as one value per person, in six windows. `phenotype_tables()` (shared helper) pairs each window with its traits:

window	feature value	traits
`training`	change T1 → T2	% change, composite, volume
`baseline`	T1 level	% change, composite, volume (does the start predict the response?)
`acute`	change T2 → T3	% change, composite, volume
`baseline_level`	T1 level	trait levels at T1
`post_level`	T2 level	trait levels at T2
`post_bout_level`	T3 level	trait levels at T2 (T3 has no phenotype)

```
pairings <- phenotype_tables(phenotype)$pairings
outcomes_of <- map(pairings, \(tt) setdiff(names(tt), "subject"))
window_of <- \(values) {
  list(
    training = within_subject_change(values, meta, "T1", "T2"),
    baseline = values_at_timepoint(values, meta, "T1"),
    acute = within_subject_change(values, meta, "T2", "T3"),
    baseline_level = values_at_timepoint(values, meta, "T1"),
    post_level = values_at_timepoint(values, meta, "T2"),
    post_bout_level = values_at_timepoint(values, meta, "T3")
  )
}
enough <- \(subjects, outcome) min(table(group_of[subjects][!is.na(outcome)])) >= 5
lengths(outcomes_of)
```

```
       training        baseline           acute  baseline_level      post_level
              8               8               8               6               6
post_bout_level
              6
```

Tests per feature type

For each feature type: every window-by-trait screen, Spearman within each group (`spearman_by_row()`, exact p), then the group-by-trait limma fit (`fit_subject_model()`, robust empirical Bayes) on the traits that did not define the grouping and have five people per group. BH and Storey's q run within level, window, trait (and group, and collection for sets).

```
per_level <- imap(feature_levels, \(feature_level, level_name) {
  windows <- window_of(feature_level$values)
  screens <- imap(windows, \(x, window) {
```

```
  map(set_names(outcomes_of[[window]]), \(name) {
    list(x = x, outcome = pairing_value(pairings, window, name, colnames(x)))
  })
})
within_group <- imap(screens, \(by_outcome, window) {
  imap(by_outcome, \(s, name) {
    map(set_names(group_levels), \(g) {
      kept <- group_of[colnames(s$x)] == g
      spearman_by_row(s$x[, kept, drop = FALSE], s$outcome[kept], exact = TRUE)
    }) |>
      list_rbind(names_to = "group") |>
      mutate(trait = name)
  }) |>
    list_rbind() |>
    mutate(window = window)
}) |>
  list_rbind()
no_fits <- tibble(
  feature = character(), coefficient = character(), n_subjects = double(),
  estimate = double(), estimate_lower = double(), estimate_upper = double(), t = double(),
  p = double(), trait = character()
)
group_difference <- imap(screens, \(by_outcome, window) {
  by_outcome[setdiff(names(by_outcome), defining_outcomes)] |>
    keep(\(s) enough(colnames(s$x), s$outcome)) |>
    imap(\(s, name) {
      predictors <- data.frame(
        group = factor(group_of[colnames(s$x)], group_levels), outcome = s$outcome
      )
      term <- sprintf("group%s:outcome", group_levels[2])
      fit_subject_model(s$x, predictors, ~ group * outcome, term) |>
        mutate(trait = name)
    }) |>
    list_rbind(ptype = no_fits) |>
    mutate(window = window)
}) |>
  list_rbind() |>
  select(-coefficient) |>
  rename(
    slope_difference = estimate, slope_difference_lower = estimate_lower,
    slope_difference_upper = estimate_upper
  )
tidy <- \(x) {
  x |>
    mutate(
      feature_type = level_name, label = feature_level$label[feature],
      collection = if (is.null(feature_level$collection)) {
        NA_character_
      } else {
        feature_level$collection[feature]
      },
```

```
        .before = feature
      ) |>
      mutate(
        fdr = p.adjust(p, "BH"), qvalue = storey_q(p),
        .by = c(feature_type, window, trait, any_of("group"), collection)
      )
  }
  list(
    within_group = tidy(filter(within_group, !is.na(rho))),
    group_difference = tidy(group_difference)
  )
})
results <- map(set_names(names(per_level[[1]])), \(sheet) list_rbind(map(per_level, sheet)))
map_int(results, nrow)
```

```
    within_group group_difference
          276612                0
```

How many links, against chance

At p < 0.05, chance alone puts 5% of features below the line in every screen.

```
counts <- imap(results, \(x, test) {
  summarise(x,
    n_tested = n(), n_nominal = sum(p < 0.05), n_fdr_05 = sum(fdr < 0.05),
    .by = c(feature_type, window, trait, any_of("group"))
  ) |>
    mutate(test = test, .before = 1)
}) |>
  list_rbind()
counts |>
  summarise(
    n_tested = sum(n_tested), n_nominal = sum(n_nominal), n_fdr_05 = sum(n_fdr_05),
    .by = test
  )
```

```
# A tibble: 1 × 4
  test         n_tested n_nominal n_fdr_05
  <chr>           <int>     <int>    <int>
1 within_group   276612      5260       21
```

```
counts |>
  mutate(
    test = if_else(is.na(group), test, paste(test, group)),
    window = factor(window, names(window_of(feature_levels$modules$values)))
  ) |>
  ggplot(aes(n_nominal / n_tested, trait, fill = test)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.75) +
```

```
  geom_vline(xintercept = 0.05, linetype = "dashed", linewidth = 0.3) +
  facet_grid(feature_type ~ window, scales = "free_y", space = "free_y") +
  scale_x_continuous(labels = scales::label_percent()) +
  labs(x = "Features at p < 0.05", y = NULL, fill = NULL) +
  theme_bw(base_size = 8) +
  theme(legend.position = "bottom")
```

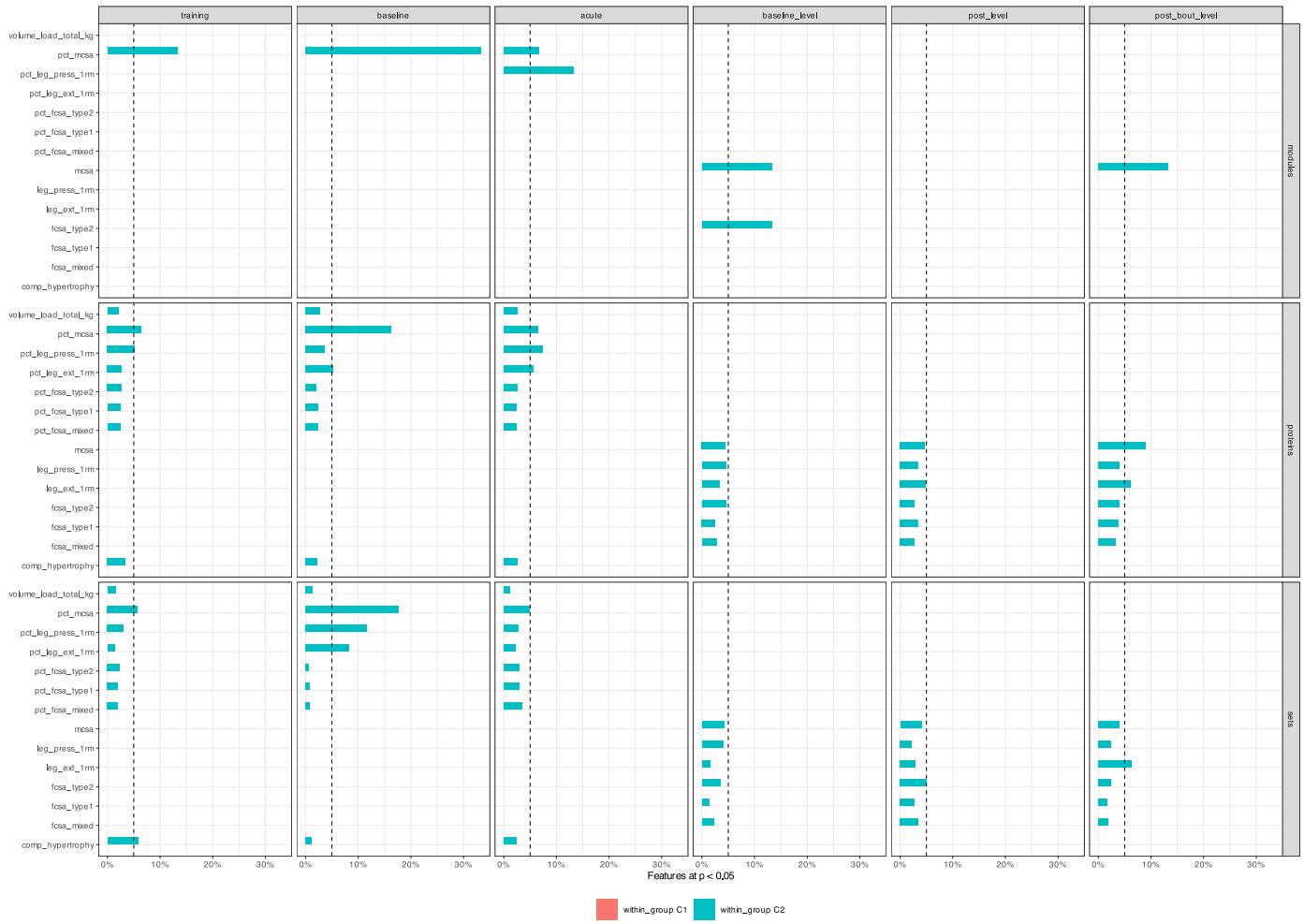


Figure 1: Share of features at p < 0.05 per test, window and trait. Dashed: 5%.

Validation

```
stopifnot(
  all(results$within_group$fdr >= results$within_group$p),
  all(abs(results$within_group$rho) <= 1),
  !any(results$group_difference$trait %in% defining_outcomes)
)
```

Summary

```
tally <- \(test_name) filter(counts, test == test_name) |> summarise(across(starts_with("n_"),
sum))
between <- tally("group_difference")
within <- tally("within_group")
```

Group difference (limma, feature ~ group × trait): 0 tests, 0 at p < 0.05, 0 at BH < 0.05. Within each group (Spearman): 276,612 tests, 5,260 at p < 0.05, 21 at BH < 0.05.

Save

```
out <- here(data_dir)
dir.create(out, recursive = TRUE, showWarnings = FALSE)
write_workbook(
  c(list(counts = counts), map(results, \(x) arrange(x, feature_type, window, trait, p))),
  c(
    "Tested, p < 0.05 and BH < 0.05 per test, feature type, window, trait and group.",
    "Spearman rho with its Fisher-z 95% CI and exact p in each group, every feature and
trait.",
    sprintf(
      paste(
        "limma, feature ~ group * trait: %s slope minus %s slope with its 95%% CI, t, p, BH
and",
        "Storey q."
      ),
      group_levels[2], group_levels[1]
    )
  ),
  file.path(out, "06_trait_associations.xlsx"), inputs, paths
)
```